04 PID-control

November 24, 2025

0.1 Import

[8]:
```
import numpy as np
import matplotlib.pyplot as plt
import tclab
import time
from IPython.display import display, clear_output
```

[90]:
```
lab.close()
```

```
TCLab disconnected successfully.
```

[101]:
```
lab = tclab.TCLab()
channel= 1   #select which heater and sensor to use, use 2 to speed up the␣
 ↪lecture and avoid cooling
```

```
TCLab version 1.0.0
Arduino Leonardo connected on port COM5 at 115200 baud.
TCLab Firmware 2.0.1 Arduino Leonardo/Micro.
```

[105]:
```
print('T1=', lab.T1, ' T2=',lab.T2) # check initial conditions
```

```
T1= 26.377  T2= 25.732
```

0.2 Usiamo il primo metodo Z-N: valutiamo la risposta ad un gradino

[ ]:
```
n = 600  # Number of second time points
tm = np.linspace(0,n-1,n) # Time values
T1 = np.zeros(n) # vector of measured temperatures
Q1 = np.zeros(n) # vector of actuated controls


# Start live plot
plt.ion()# Turn on matplotlib interactive mode
fig, axs = plt.subplots(2, 1, figsize=(20, 14))

# simulation loop

for i in range(n): # for all time steps
```

```
    # record measurement
    if channel==1:
        T1[i] = lab.T1
    else:
        T1[i] = lab.T2


    Q1[i]=100


    # Actuate control
    if channel==1:
        lab.Q1(Q1[i])
    else:
        lab.Q2(Q1[i])

    # --- Live updating plot ---
    clear_output(wait=True)

    # Subplot 0
    axs[0].cla()
    # Set the background color of the plot area to white
    axs[0].set_facecolor('white')
    axs[0].grid(True, which='major', linestyle='-', linewidth=0.1,␣
↪color='gray') # Customize grid
    axs[0].plot(tm[:i+1]/60.0, T1[:i+1], 'r.', label=r'$T_1$ PV')
    axs[0].set_ylabel(r'Temp ($^oC$)')

    # Set grid intervals for axs[0] (X-axis: 0 to n/60, Y-axis: 15 to 100)
    axs[0].set_xlim([0, n/60])
    axs[0].set_ylim([20, 120])
    axs[0].set_xticks(np.arange(0, n/60 + 0.01, 0.2))
    axs[0].set_yticks(np.arange(20, 120.01, 3))

    axs[0].legend(loc=2)

    # Subplot 1
    axs[1].cla()
    # Set the background color of the plot area to white
    axs[1].set_facecolor('white')
    axs[1].grid(True, which='major', linestyle='-', linewidth=0.1,␣
↪color='gray') # Customize grid
    axs[1].plot(tm[:i+1]/60.0, Q1[:i+1], 'b-', label=r'$Q_1$ actuated')
    axs[1].set_ylabel(r'Heater (%)')
    axs[1].set_xlabel('Time (min)')
```

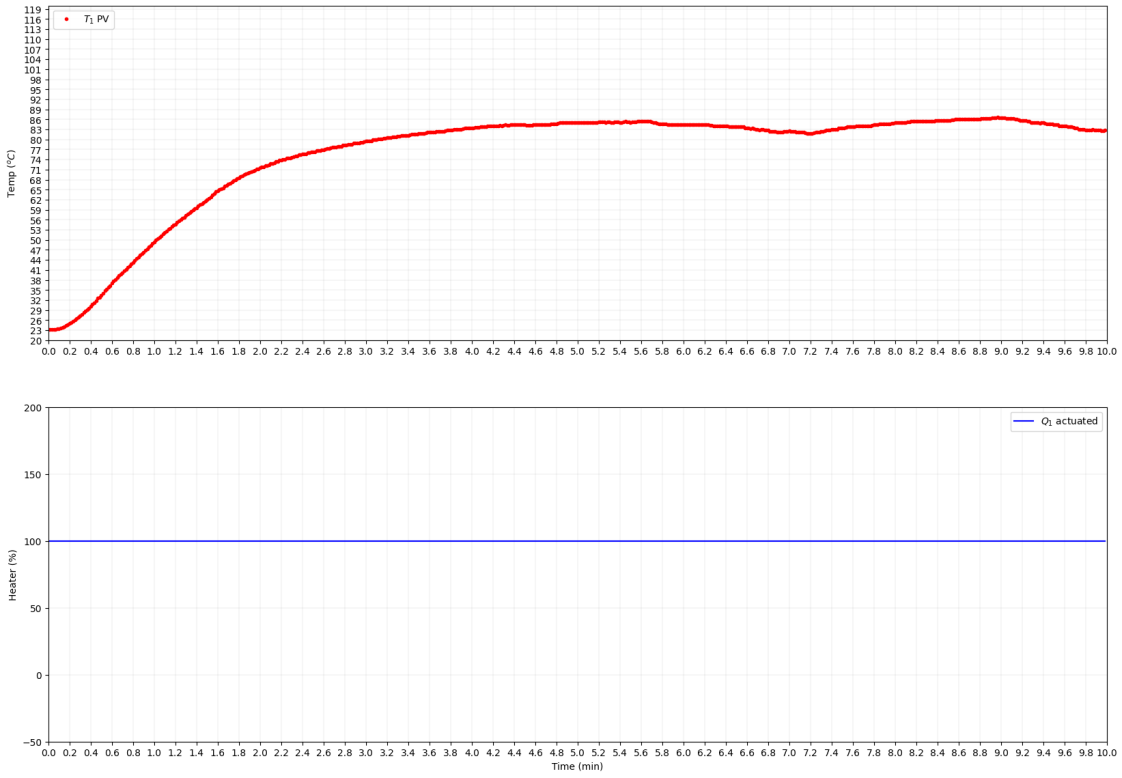
```
    # Set grid intervals for axs[1] (X-axis: 0 to n/60, Y-axis: -50 to 200)
    axs[1].set_xlim([0, n/60])
    axs[1].set_ylim([-50, 200])
    axs[1].set_xticks(np.arange(0, n/60 + 0.01, 0.2))

    axs[1].legend(loc=1)

#    fig.tight_layout()
    display(fig)
    plt.pause(0.01)
    time.sleep(1)

plt.ioff()  # Turn off interactive mode
lab.close()
```

[99]:



0.3 Parametri simulazione

[106]:
```
# ----------------------------
# Evaluate controller gains
# ----------------------------


# Identified parameters
k = 0.65
tau = 108
theta = 12.0

# ----------------------------
# Calculate Kc,tauI,tauD (Z-N first method)
# ----------------------------
theta_d=theta+1/2

Kc=1/k*1.2*(tau/theta_d)
tauI=2*theta_d
tauD=0.5*theta_d

print('Kc: ' + str(Kc))
print('tauI: ' + str(tauI))
print('tauD: ' + str(tauD))
```

```
Kc: 15.950769230769229
tauI: 25.0
tauD: 6.25
```

0.4 Avvio simulazione

[130]:
```
lab = tclab.TCLab()
channel= 2   #select which heater and sensor to use, use 2 to speed up the␣
 ↪lecture and avoid cooling
```

```
TCLab version 1.0.0
Arduino Leonardo connected on port COM5 at 115200 baud.
TCLab Firmware 2.0.1 Arduino Leonardo/Micro.
```

[134]:
```
print('T1=', lab.T1, ' T2=',lab.T2) # check initial conditions
```

```
T1= 27.021  T2= 24.443
```

0.5 definizione funzione PID_y

[135]:
```
#------------------------
# PID Controller Function
#------------------------
# sp = setpoint
# pv = current temperature
# pv_last = prior temperature
# ierr = integral error
# dt = time increment between measurements
# outputs ---------------
# op = output of the PID controller
# P = proportional contribution
# I = integral contribution
# D = derivative contribution
def pid(sp,pv,pv_last,ierr,dt):
    # Parameters in terms of PID coefficients
    KP = Kc
    KI = Kc/tauI
    KD = Kc*tauD
    # ubias for controller (initial heater)
    op0 = 0
    # upper and lower bounds on heater level
    ophi = 100
    oplo = 0
    # calculate the error
    error = sp-pv
    # calculate the integral error
    ierr = ierr + KI * error * dt
    # calculate the measurement derivative
    dpv = (pv - pv_last) / dt
    # calculate the PID output
    P = KP * error
    I = ierr
    D = -KD * dpv
    op = op0 + P + I + D
    # implement anti-reset windup
    if op < oplo or op > ophi:
        I = I - KI * error * dt
        # clip output
        op = max(oplo,min(ophi,op))
    # return the controller output and PID terms
    return [op,P,I,D]
```

0.5.1 Loop simulazione e plot

[136]:
```
# Start live plot
plt.ion()# Turn on matplotlib interactive mode
fig, axs = plt.subplots(2, 1, figsize=(20, 14))

h_ierr=np.zeros(n) # to store data for prints
h_D=np.zeros(n)
h_P=np.zeros(n)


n = 180  # Number of second time points
tm = np.linspace(0,n-1,n) # Time values



T1 = np.zeros(n) # vector of measured temperatures
Q1 = np.zeros(n) # vector of actuated controls
# step setpoint from 23.0 to 40.0 degC
set_point= 40.0
SP1 = np.ones(n)*23.0 # 23 is assumed as air temperature and reference value␣
 ↪for the first 10 seconds
SP1[10:] = set_point
Q1_bias = 0.0
ierr = 0.0 # initial level of integral action
ierr=0


# simulation loop

for i in range(n): # for all time steps


    # record measurement
    if channel==1:
        T1[i] = lab.T1
    else:
        T1[i] = lab.T2

    # ------------------------------------------------
    # call PID controller function to change Q1[i]
    # ------------------------------------------------
    [Q1[i],P,ierr,D] = pid(SP1[i],T1[i],T1[max(0,i-1)],ierr,1.0)

    h_P[i]=P
    h_ierr[i]=ierr
    h_D[i]=D
```

```
    # Actuate control
    if channel==1:
        lab.Q1(Q1[i])
    else:
        lab.Q2(Q1[i])

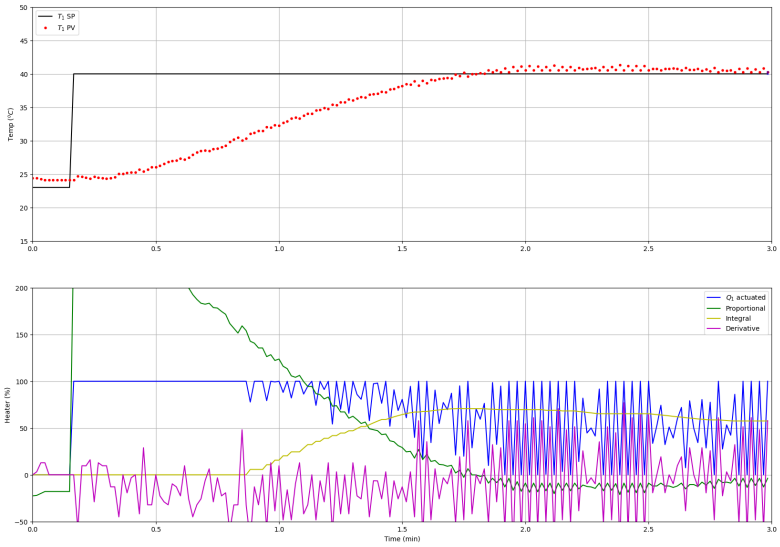
    # --- Live updating plot ---
    clear_output(wait=True)
    axs[0].cla()
    axs[1].cla()
    axs[0].grid()
    axs[0].plot(tm[:i+1]/60.0, SP1[:i+1], 'k-', label=r'$T_1$ SP')
    axs[0].plot(tm[:i+1]/60.0, T1[:i+1], 'r.', label=r'$T_1$ PV')
    axs[0].text(4.1, 30, 'Measured Offset: ' + str(np.round(SP1[i]-T1[i],2)))
    offset = set_point - (23 + 0.9*Kc*set_point) / (1 + 0.9*Kc)
    axs[0].text(4.1, 26, 'Calculated Offset: ' + str(np.round(offset,2)))
    axs[0].text(4.1, 22, r'$K_c$: ' + str(np.round(Kc,2)))
    axs[0].text(4.1, 18, r'$K_i$: ' + str(np.round(Kc/tauI,2)))

    axs[0].plot([tm[i]/60.0, tm[i]/60.0], [SP1[i], T1[i]], 'b-', lw=3, alpha=0.
 ↪5)
    axs[0].set_ylabel(r'Temp ($^oC$)')
    axs[0].set_xlim([0, n/60])
    axs[0].set_ylim([15, np.max(SP1)*1.25])
    axs[0].legend(loc=2)

    axs[1].grid()
    axs[1].plot(tm[:i+1]/60.0, Q1[:i+1], 'b-', label=r'$Q_1$ actuated')
    axs[1].plot(tm[:i+1]/60.0, h_P[:i+1], 'g-', label=r'Proportional')
    axs[1].plot(tm[:i+1]/60.0, h_ierr[:i+1], 'y-', label=r'Integral')
    axs[1].plot(tm[:i+1]/60.0, h_D[:i+1], 'm-', label=r'Derivative')


    axs[1].set_ylabel(r'Heater (%)')
    axs[1].set_xlabel('Time (min)')
    axs[1].legend(loc=1)
    axs[1].set_xlim([0, n/60])
    axs[1].set_ylim([-50,200])
#    fig.tight_layout()
    display(fig)
    plt.pause(0.01)
    time.sleep(1)

plt.ioff()  # Turn off interactive mode
lab.close()
```



Measured Offset: -0.23

Calculated Offset: 1.11

K_c: 15.95

K_i: 0.64

```
TCLab disconnected successfully.
```

0.5.2 Sicuramente c'è margine di miglioramento sul derivativo, ma il PID_y è completo!

[129]:
```
lab.close()
```

```
ERROR! Session/line number was not unique in database. History logging moved to
new session 15
TCLab disconnected successfully.
```

[ ]:

[ ]:

[ ]:

[ ]: